Report2--BombLab

一、实验概览

运行环境：WSL2 Ubuntu 24.04，uname -sm 输出 Linux x86_64；使用 GCC、GNU Make、Python 3、GDB 15.1 与 binutils。

完成情况：六个正式关卡与 Honor 全部通过。

自测结果：Public tests: 60/60, Secret public tests: 10/10。

```
jackyning@jackyning:~/ics-2026-bomblab$ uname -sm
Linux x86_64
```

本实验提交的是解法程序，不是六个固定密码。每关会给出一个数值 challenge，正确口令随它变化。因此下面每一关的重点都是恢复出对任意 challenge 都成立的计算关系，而不是记录某一次运行时的比较值。

二、分析方法

分析过程中交替使用两种方式。

（一）离线分析

把目标函数导成汇编，在编辑器中阅读、标注：

```
objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb > phase_01.S
```

如果关卡调用了其他函数，同样要导出并阅读，而不是只看入口函数。

（二）在线核对

用 GDB 在关键比较指令处停下，读取寄存器与栈帧中的实际值，再用推测出的公式复算，与程序算出的期望值比对。两者相等，说明从汇编读出的关系是对的。

```
(gdb) break *phase_01
(gdb) run --seed 42 --phase 1 < input.txt
(gdb) p/x $rsi
```

（三）两点统一约定

第一，六个关卡入口都有 mov %rsi,%r12，即 challenge 被保存在 %r12 中，之后一律从 %r12 读取；输入串则由 mov %rdi,%rbx 保存。

第二，六关的结构高度一致，读法可以固定成三步。先看 mov $N,%esi 中的立即数 N，它是 valid_u32_fields 的第二个参数，也就是本关需要的整数个数。再看若干个 cmp ...,-0x??(%rbp)，这些是逐项校验，左操作数是程序算出的期望值，右操作数是输入。最后看结尾的 sete %al，它把比较结果直接作为返回值，成功为 1。

三、正式六关

（一）第 1 关：函数调用与参数传递

1. 关键汇编

phase_01（0x402090）原文摘录：

```
40209d: mov    $0x1,%esi              # valid_u32_fields(s, 1)，只要 1 个整数
4020aa: call   402c80 <valid_u32_fields>
4020c2: lea    -0x14(%rbp),%rdx       # sscanf 的第 3 个参数：&v0
4020ce: call   4011e0 <__isoc99_sscanf@plt>
4020d3: cmp    $0x1,%eax
4020d8: mov    %r12,%rdi              # rdi = challenge
```

```
4020db: call   402050 <p1_compute>
4020e0: cmp    %eax,-0x14(%rbp)       # 输入 v0 与 p1_compute(challenge) 比较
4020e3: sete   %al
```

2. 恢复出的计算关系

```
uint32_t p1_compute(uint64_t ch)
{
    uint32_t lo  = (uint32_t)ch;          /* 低 32 位 */
    uint32_t hi  = (uint32_t)(ch >> 32);  /* 高 32 位 */
    uint32_t mid = (uint32_t)(ch >> 16);  /* bits 16 起 */
    uint32_t a = (lo + 0xfe5eu) ^ hi;
    uint32_t b = (mid ^ 0xa4afu) + lo * 2u;
    return (b - a) * 3u + 0xd180u;
}
```

口令为 1 个整数，等于 p1_compute(challenge)。

3. 推理过程

（1）mov %rsi,%r12 与 mov %rdi,%rbx 确定了 challenge 与输入串各自的位置。

（2）mov $0x1,%esi 直接给出整数个数为 1，随后 cmp $0x1,%eax 与 sscanf 的返回值吻合。

（3）全关只有一处比较 cmp %eax,-0x14(%rbp)，左操作数是 p1_compute 的返回值，右操作数是刚解析出的输入，因此口令必然等于该函数对 challenge 的输出。

（4）继续导出 p1_compute：它把 challenge 拆成低 32 位、高 32 位和 bits 16 起的三段，依次做"加立即数后异或""异或后加二倍自身""相减后乘 3 再加立即数"三步，得到一个 32 位结果。所有运算都是 32 位宽度，会按模 2 的 32 次方回绕。

（二）第 2 关：数组、循环与递推关系

1. 关键汇编

phase_02（0x4020f0）原文摘录：

```
4020fd: mov    $0x6,%esi              # 6 个整数
40210c: call   402c80 <valid_u32_fields>
402155: cmp    $0x6,%eax              # sscanf 必须解析出 6 个字段
40215a: mov    %r13,%rax
402160: shr    $0x24,%rax             # challenge >> 36
402164: and    $0x3f,%eax             # & 0x3F
402167: lea    (%rax,%rax,2),%eax     # 乘 3
40216a: cmp    %eax,%esi              # 校验 v[0]
40216e: mov    $0x1,%edx              # i = 1
402173: lea    0xa(%rdx),%ecx         # 移位量 = i + 10
402176: mov    %r13,%rax
402179: shr    %cl,%rax               # challenge >> (i + 10)
40217c: lea    (%rsi,%rsi,4),%ecx     # 5 * v[i-1]
40217f: mov    -0x40(%rbp,%rdx,4),%esi  # v[i]
402183: and    $0xf,%eax              # & 0xF
402186: add    %edx,%eax              # + i
402188: xor    %ecx,%eax              # ^ (5 * v[i-1])
```

```
40218a: cmp    %eax,%esi              # 校验 v[i]
40218e: add    $0x1,%rdx              # i++
402192: cmp    $0x6,%rdx
402196: jne    402173                 # 循环到 i = 5
```

2. 恢复出的计算关系

```
v[0] = 3u * (uint32_t)((challenge >> 36) & 0x3F);
for (i = 1; i < 6; ++i) {
    uint32_t t = (uint32_t)((challenge >> (i + 10)) & 0xF);
    v[i] = (t + i) ^ (5u * v[i - 1]);
}
```

口令为 6 个整数 v[0] 到 v[5]。

3. 推理过程

（1）mov $0x6,%esi 与 cmp $0x6,%eax 双重确认有 6 个整数。

（2）v[0] 与 v[1..5] 使用两段不同的式子，所以是"首项递推"结构而不是一条统一公式。

（3）lea (%rax,%rax,2),%eax 是 x + 2x = 3x 的编译器惯用写法，对应首项的乘 3。

（4）循环里 lea 0xa(%rdx),%ecx 把 i + 10 放进 %cl，紧接着 shr %cl,%rax，说明移位量随下标变化，这是"每个下标从 challenge 取不同的 4 位"的直接证据；and $0xf 说明只取 4 位。

（5）lea (%rsi,%rsi,4),%ecx 是乘 5，此时 %esi 持有上一轮的 v[i-1]；xor %ecx,%eax 说明递推形如"新值等于 challenge 的函数异或 5 倍前一项"。

（6）数组基址 -0x40(%rbp) 配合 %rdx 的比例变址（下标乘 4，正好是 uint32_t 的宽度）说明这 6 个整数就是栈上的一个 uint32_t 数组。

（7）循环上界 cmp $0x6,%rdx 加 jne 给出 i 从 1 到 5，共 5 次迭代。

（三）第 3 关：switch 分支与跳转表

1. 关键汇编

phase_03 (0x4022a0) 原文摘录：

```
4022ad: mov    $0x2,%esi              # 2 个整数
4022d0: lea    -0x14(%rbp),%rcx       # &v1
4022d2: lea    -0x18(%rbp),%rdx       # &v0
4022e2: call   __isoc99_sscanf@plt
4022e7: cmp    $0x2,%eax
4022ec: mov    %r12d,%edi             # 低 32 位
4022f1: and    $0x7,%edi              # sel = challenge & 7
4022f4: cmp    %edi,-0x18(%rbp)       # 校验 v0 == sel
4022f9: mov    %r12,%rax
4022fc: mov    %r12,%rsi
4022ff: shr    $0x3,%rax              # challenge >> 3
402303: shr    $0x20,%rsi             # challenge >> 32
402307: xor    %eax,%esi              # x = 高 32 位 ^ (challenge >> 3)
402309: call   402240 <p3_dispatch>
40230e: cmp    %eax,-0x14(%rbp)       # 校验 v1
```

2. 恢复出的计算关系

```
uint32_t sel = (uint32_t)challenge & 7u;          /* v0 */
uint32_t x   = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);
uint32_t v1  = p3_dispatch(sel, x);
```

口令为 sel 与 v1 两个整数。

p3_dispatch 内部是 jmp *0x405xxx(,%rax,8) 形式的跳转表，8 个表项对应 8 个分支。逐分支读出的运算如下：

```
sel = 0:  (x ^ 0xba19) + (x >> 2)
sel = 1:  5x + 0x42b8
sel = 2:  5x + 0x3f601
sel = 3:  (2x) ^ 0xbc9
sel = 4:  5x + 0x3728
sel = 5:  (x ^ 0xf30) + (x >> 1)
sel = 6:  (4x) ^ 0xb150
sel = 7:  5x + 0x79e2
```

3. 推理过程

（1）先看到 and $0x7 只保留低 3 位，紧接一次比较，说明第一个数是分支选择子，取值 0 到 7。

（2）mov %r12d,%edi 是 32 位写，会把 %rdi 的高 32 位清零，所以传给 p3_dispatch 的第一个参数是干净的 32 位值。

（3）x 由 challenge >> 32 与 challenge >> 3 异或得到，两者一个取高位、一个取中位，保证 8 个分支都能拿到不同的输入。

（4）p3_dispatch 里 jmp *0x405xxx(,%rax,8) 是跳转表的典型形态，表项 8 字节，直接存代码地址。用 GDB 的 x/8gx 把表打出来，再逐个导出目标地址，就得到上面 8 条公式

（四）第 4 关：递归调用与返回过程

1. 关键汇编

phase_04（0x402390）原文摘录：

```
40239d: mov    $0x1,%esi              # 1 个整数
4023c2: lea    -0x14(%rbp),%rdx       # &v0
4023ce: call   __isoc99_sscanf@plt
4023d3: cmp    $0x1,%eax
4023d8: mov    %r12,%rcx
4023db: mov    %r12,%rax
4023de: mov    %r12d,%edi
4023e1: xor    %esi,%esi              # 第 2 个参数 = 0
4023e3: shr    $0x20,%rax             # challenge >> 32
4023e7: shr    $0x7,%rcx              # challenge >> 7
4023eb: and    $0x7f,%edi             # 第 1 个参数 = challenge & 0x7F
4023ee: mov    $0x7f,%edx             # 第 3 个参数 = 0x7F
4023f3: xor    %eax,%ecx              # 第 4 个参数 = (ch>>32) ^ (ch>>7)
4023f5: call   402320 <p4_walk>
4023fa: cmp    %eax,-0x14(%rbp)
```

2. 恢复出的计算关系

```
uint32_t p4_walk(uint32_t a, uint32_t b, uint32_t c, uint32_t d)
{
    uint32_t mid = b + ((c - b) >> 1);
    uint32_t key = mid + d;
    if (a == mid) return (mid + 0xd6u) ^ d;
    if (a <  mid) return (5u * p4_walk(a, b, mid - 1u, d)) ^ key ^ 0x13u;
                  return (5u * p4_walk(a, mid + 1u, c, d)) ^ key ^ 0x09u;
}
v0 = p4_walk((uint32_t)challenge & 0x7Fu, 0u, 0x7Fu,
             (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 7));
```

3. 推理过程

（1）按 System V AMD64 约定，四个参数依次落在 %rdi、%rsi、%rdx、%rcx，从这四条赋值可以直接读出初值：a 为 challenge & 0x7F，b 为 0，c 为 0x7F，d 为 (ch>>32) ^ (ch>>7)。

（2）and $0x7f 与 mov $0x7f,%edx 一起说明搜索区间固定为 0 到 0x7F，a 是其中的目标值。

（3）p4_walk 内部有 call p4_walk（自调用），并且返回值被乘 5 后与上层结果异或，这是递归的典型形态：先递归到底，再沿调用栈回溯时做"5 乘子结果、异或 key、异或常量"。

（4）递归出口是 a == mid，两个分支分别对应区间左半与右半。b + ((c - b) >> 1) 是无溢出取中点的写法，先减后加，而不是 (b + c) / 2，避免加法溢出。

（五）第 5 关：结构体布局、指针与链表遍历

1. 关键汇编

```
phase_05 (0x402480) 原文摘录：
40248d: mov    $0x2,%esi              # 2 个整数
4024b2: lea    -0x18(%rbp),%rcx       # &v1
4024b6: lea    -0x1c(%rbp),%rdx       # &v0
4024c2: call   __isoc99_sscanf@plt
4024c7: cmp    $0x2,%eax
4024cc: lea    -0x14(%rbp),%rsi       # 第 2 个参数是 &aux，局部变量地址
4024d0: mov    %r12,%rdi              # 第 1 个参数是 challenge
4024d3: call   402410 <p5_traverse>
4024d8: mov    %eax,%edx
4024dc: cmp    %edx,-0x1c(%rbp)       # 校验 v0 等于返回值
4024e1: mov    -0x14(%rbp),%eax
4024e4: cmp    %eax,-0x18(%rbp)       # 校验 v1 等于 *aux
```

2. 恢复出的计算关系

```
idx = (uint32_t)(challenge & 7);            /* 从哪个链表头出发 */
n   = (uint32_t)((challenge >> 3) & 3);     /* 再走几步 */
v   = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
v ^= 0xae9a;
v  = 3u * v + p5_b[idx];
v ^= p5_a[idx];
```

```
*aux = p5_a[idx];
for (i = 0; i < n; ++i) {
    idx++;
    v = 3u * v + p5_b[idx];
    v ^= p5_a[idx];
    *aux = p5_a[idx];
}
```

口令为返回值 v 与 *aux 两个整数。

链表节点的实际数据（按链表物理顺序，从 .data 段 0x405ba0 起）：

```
static const uint32_t p5_a[12] = {
    0x0b15, 0x0638, 0x090a, 0x0a7b, 0x005b, 0x043e,
    0x0853, 0x0206, 0x051c, 0x0159, 0x0334, 0x0734
};
static const uint32_t p5_b[12] = {
    0xe0a0, 0x92ad, 0x2b91, 0xad7d, 0xa0eb, 0x6a61,
    0x63f5, 0x9a5d, 0xca9a, 0xafe7, 0xc1f5, 0xe511
};
```

3. 推理过程

（1）第二个参数是 lea -0x14(%rbp),%rsi，传入的是一个局部变量的地址，说明函数除了返回值还要带出第二个量，这与两个整数的口令格式正好对应。

（2）and $7 给出 8 种起点，(challenge >> 3) & 3 给出 0 到 3 步的步数，这是"从某个链表头出发、沿 next 指针走 n 步"的形态。

（3）用 x/48wx 0x405ba0 按 4 字节粒度把节点数据打出来。先用 8 字节粒度读时，0x0000063f50000853 这种打包值会把 a、b 两个字段糊在一起，必须换小粒度才能分开，最终得到 12 个节点的 a、b 字段。

（4）再用 x/8gx 读 8 个链头，它们的地址恰好落在同一个 12 节点数组的不同下标上，说明 8 个"链表"其实是同一条链表的 8 个后缀。因此不必写 8 条链，用一个 12 元素的数组加下标推进即可，这也让 idx++ 的写法有了正确含义。

（5）循环体先算 v = 3v + b[idx] 再算 v ^= a[idx]；*aux 每步被覆盖成当前的 a[idx]，所以循环结束后 *aux 是最后一个访问节点的 a 字段。

（六）第 6 关：函数指针表与间接调用

1. 关键汇编

phase_06（0x4025d0）原文摘录：

```
4025dd: mov    $0x1,%esi              # 1 个整数
402602: lea    -0x14(%rbp),%rdx       # &v0
40260e: call   __isoc99_sscanf@plt
402613: cmp    $0x1,%eax
402618: mov    %r12,%rdi              # rdi = challenge
40261b: call   402580 <p6_dispatch>
402620: cmp    %eax,-0x14(%rbp)       # 校验 v0
```

2. 恢复出的计算关系

```
i1  = (uint32_t)(challenge & 7);          /* 第一次调用的函数下标 */
i2  = (uint32_t)((challenge >> 3) & 7);   /* 第二次调用的函数下标 */
arg = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
r1 = op[i1](arg);
r2 = op[i2](r1 ^ 0x5341);
v0 = r2 + 0xbe24;
```

函数指针表有 8 个表项，但只有 6 个不同的函数（存在重复表项）：

```
idx = 0:   ((x >> 7) ^ x) + 0x5192
idx = 1:   rol(x, 7) ^ 0x4801
idx = 2:   (4x + 0xb61) ^ (5x)
idx = 3:   rol(x, 7) ^ 0x4801        （与 idx = 1 相同）
idx = 4:   (x ^ 0x1117) + (x << 7)
idx = 5:   ror(x, 2)
idx = 6:   ror(x, 2)                 （与 idx = 5 相同）
idx = 7:   (x + 0x6e41) ^ (x >> 7)
```

3. 推理过程

（1）与第 3 关同样先 and $0x7，也是 8 路选择，但这次不是跳转表，而是 call *0x405xxx(,%rax,8)，即函数指针表，表项里存的是函数地址。

（2）连续两次 and $7 分别取自 challenge 的第 0 到 2 位与第 3 到 5 位，说明一关里要调用两个不同的算子：先用一个处理原始参数，再用另一个处理中间结果。

（3）用 x/8gx 打出函数指针表，会发现 8 个地址里只有 6 个互不相同（1 与 3 相同、5 与 6 相同），所以逐条导出 6 个函数即可，不必写 8 份。

（4）rol 与 ror 在汇编里由 rol $0x7 与 ror $0x2 直接给出；C 里要写成 (x << 7) | (x >> 25) 和 (x >> 2) | (x << 30)，用 uint32_t 保证移位数合法。

（5）两次调用之间夹一个 xor $0x5341，最后再加 0xbe24，这两处是关卡设计的串联常数。

四、Honor

Honor 的入口是 secret_gate，在六关全部通过且第七行非空时被调用。它依次调用 secret_copy 与 secret_phase。

（一）secret_gate：长度检查与缓冲区布局

```
40274b: lea    -0xd0(%rbp),%r12       # 缓冲区 buf = rbp - 0xd0
402737: mov    $0x14,%ecx             # rep stos 20 个 qword = 160 字节
402760: rep stos %rax,%es:(%rdi)
402763: movl   $0x0,(%rdi)            # 再清 4 字节，合计 164 字节
40276c: call   strlen
402771: sub    $0x24,%rax             # strlen(line) - 36
402775: cmp    $0x7f,%rax
402779: jbe    402790                 # 要求 (len - 36) <= 127，即 36 <= len <= 163
402796: call   secret_copy            # strcpy(buf, line)
40279b: cmpl   $0x4e45504f,-0xb0(%rbp)   # buf + 32 处必须是 "OPEN"
4027a5: jne    fail
```

```
4027ad: lea    -0xac(%rbp),%rdi       # rdi = buf + 36
4027b4: call   secret_phase
```

关键观察：buf 在 rbp − 0xd0，而检查地址是 rbp − 0xb0，两者相差 0x20 即 32 字节；secret_phase 的参数是 rbp − 0xac，即 buf + 36。因此 "OPEN" 位于偏移 32，而不是开头。

长度上限也印证了这个布局：缓冲区共清零 160 加 4，合计 164 字节，而 len 的上限是 163，加上结尾的 '\0' 正好 164，精确闭合。下限 36 等于 32 加 4，正好覆盖到偏移 32 处的 4 字节标记。

（二）secret_copy：就是 strcpy

```
402714: xor    %eax,%eax
402720: movzbl (%rsi,%rax,1),%edx     # dl = src[i]
402724: mov    %dl,(%rdi,%rax,1)      # dst[i] = dl
402727: add    $0x1,%rax
40272b: test   %dl,%dl
40272d: jne    402720                 # 非 '\0' 就继续
40272f: ret
```

逐字节复制、遇 '\0' 停止，没有任何变换。所以第七行必须字面上在偏移 32 处写出 OPEN。

（三）valid_u32_fields：严格的格式校验

secret_phase 首先调用 valid_u32_fields(s, 3)。其中 testb $0x20,0x1(%rsi,%rdx,2) 测的是 glibc __ctype_b_loc() 表项的高字节第 0x20 位，即 _ISspace；0xcccccccd 配合 shr $0x23 是编译器生成的 uint32 / 10。

还原后的语义为：

```
int valid_u32_fields(const char *s, int n)
{
    int count = 0;
    for (;;) {
        while (isspace((unsigned char)*s)) s++;
        if (*s < '0' || *s > '9') return 0;
        uint32_t acc = 0;
        for (;;) {
            uint32_t d = *s - '0';
            if (acc > (0xFFFFFFFFu - d) / 10u) return 0;   /* 溢出即拒绝 */
            acc = acc * 10u + d;
            s++;
            if (*s < '0' || *s > '9') break;
        }
        if (*s != '\0' && !isspace((unsigned char)*s)) return 0;
        if (++count == n) break;
    }
    while (isspace((unsigned char)*s)) s++;
    return *s == '\0';                                  /* 不许有尾巴 */
}
```

即：恰好 3 个以空白分隔的十进制无符号整数，不允许正负号、不允许溢出、不允许其他字符。

（四）secret_mix 与 secret_phase

```
402634: mov    %edi,%eax
402636: rol    $0x5,%eax              # 循环左移 5 位
402639: add    $0xb5c9,%eax
40263e: xor    $0x9b19,%eax
402643: ret
```

```
secret_mix(x) = (rol32(x, 5) + 0xb5c9u) ^ 0x9b19u;
```

注意它只写 %eax，不碰 %esi；这一点很重要，因为 secret_phase 在 call secret_mix 之后仍然使用 %esi（见下面的 shr $0x5,%esi）。

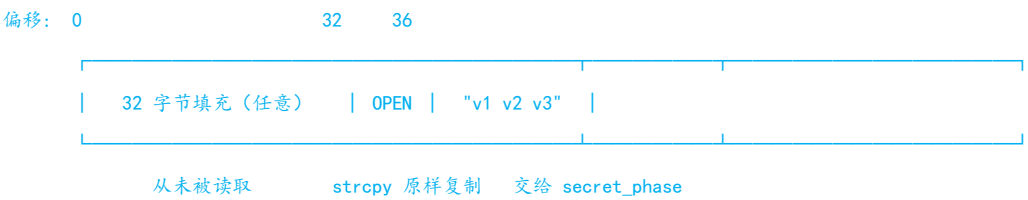
secret_phase(s, A, B)（A 在 %rsi，B 在 %rdx）还原为：

```
uint32_t c    = (uint32_t)B & 3u;
uint32_t amix = (uint32_t)(A >> 32) ^ (uint32_t)A;
uint32_t bmix = (uint32_t)(B >> 32) ^ (uint32_t)B;
uint32_t k    = (c * 0xb5c9u) ^ (uint32_t)(B >> 16);
uint32_t m    = secret_mix(amix ^ bmix);
uint32_t v2   = m ^ k;
uint32_t v3   = v2 ^ (bmix + c + 0x9b19u + (amix >> 5));
```

其中 v1 等于 c，且必须满足 v1 与 B & 3 相等。对应指令：

```
4026b5: and    $0x3,%ecx              # c = B & 3
4026b8: cmp    %ecx,-0x2c(%rbp)       # v1 == c
4026c0: imul   $0xb5c9,%ecx,%r9d
4026d8: shr    $0x10,%r13             # B >> 16
4026de: xor    %r13d,%r9d             # k
4026e4: call   secret_mix
4026e9: xor    %r9d,%eax
4026ec: cmp    %eax,-0x28(%rbp)       # v2
4026f1: shr    $0x5,%esi              # amix >> 5
4026f4: lea    0x9b19(%r8,%rcx,1),%edx
4026fc: add    %esi,%edx
4026fe: xor    %edx,%eax
402700: cmp    %eax,-0x24(%rbp)       # v3
402703: sete   %dl
```

（五）第七行的最终格式



前 32 字节是纯伪装：secret_phase 的字符串参数是 buf + 36，valid_u32_fields 与 sscanf 都只从这里开始读。实际输出示例（seed 2025）：

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN0 1007048925 1729094375

```
./defuser --seed 2025 > /tmp/p.txt
./defuser --secret --seed 2025 > /tmp/s.txt
cat /tmp/p.txt /tmp/s.txt > /tmp/all.txt

cat /tmp/all.txt
2902827180
12 58 294 1458 7274 36353
0 113572827
2332992665
3328233413 518
1316920328
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN0 1007048925 1729094375
```

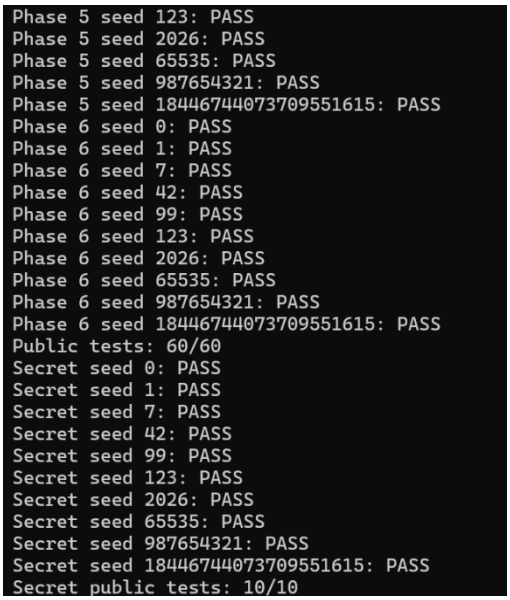
五、验证记录

（一）公开测试全量

Public tests: 60/60

Secret public tests: 10/10

```
make test
python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
```



（二）非公开 seed 验证（seed = 2025）

公开 seed 列表为 0、1、7、42、99、123、2026、65535、987654321、18446744073709551615，2025 不在其中。按 public_test.py 内部相同的调用方式复现：

```
./defuser --seed 2025 > /tmp/p.txt
./defuser --secret --seed 2025 > /tmp/s.txt
cat /tmp/p.txt /tmp/s.txt > /tmp/all.txt
./bomb --seed 2025 < /tmp/all.txt
```

结果如下表。

```
Phase 1:  Phase 1 defused.
Phase 2:  Phase 2 defused.
Phase 3:  Phase 3 defused.
Phase 4:  Phase 4 defused.
Phase 5:  Phase 5 defused.
Phase 6:  Phase 6 defused.
Honor  :  Secret phase defused.
```

```
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xfcb3a4d54e7a52c2
Input (unsigned decimal integers):
Phase 1 defused.

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

```
第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x72b5644772472ea4
Input (unsigned decimal integers):
Phase 2 defused.

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]
```

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x2d942d47fd1ccf80
Input (unsigned decimal integers):
Phase 3 defused.

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]
```

```
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xab9627bdce21e915
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]
```

```
第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0xa3ba32b100fd3b61
Input (unsigned decimal integers):
Phase 6 defused.

终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
```

```
Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

六、参考资料

- Randal E. Bryant, David R. O'Hallaron. 《深入理解计算机系统》(CS:APP，第 3 版）第 3 章「程序的机器级表示」。用于寄存器使用约定、栈帧、跳转表、函数指针与递归的分析。
- GDB 官方文档，https://www.sourceware.org/gdb/documentation/
- CS:APP x86-64 GDB 速查表，http://csapp.cs.cmu.edu/3e/docs/gdbnotes-x86-64.pdf
- System V Application Binary Interface, AMD64 Architecture Processor Supplement。用于参数寄存器与被调用者保管寄存器的划分。
- glibc <ctype.h> 中 __ctype_b_loc() 与 _ISbit 宏的定义。用于识别 valid_u32_fields 中的 isspace 判断。
- objdump 手册（man objdump)。

七、辅助工具使用说明

（1）阅读汇编：协助逐条翻译指令、识别编译器惯用写法，例如 lea (%rax,%rax,2) 表示乘 3，0xcccccccd 配合右移 35 位表示除以 10。

（2）核对计算关系：对每一关先由人工从汇编推出候选公式，再用 GDB 停在实际比较处核对。

（3）整理报告：协助组织文字与结构。

所有结论均以实测为准。每一关的实现都由 make 重新编译后，经 ./defuser --seed N | ./bomb --seed N、python3 public_test.py、make test 三层验证。报告中引用的汇编均为本机 objdump 导出的原文摘录。